FROM FIRST EXPLANATION TO TECHNICAL DEEP DIVE

NEXUS Project Interview Handbook

Learn the business problem, understand the implementation, defend the design decisions and demonstrate the complete supply-chain simulation.

32 chapters **60 interview answers** **9 worked exercises** **7 day study plan**

> PDF study edition with clickable contents and links. Read the explanations, practice the exercises, and rehearse the project demonstration. Interactive browser-only controls are available in the HTML edition.

Reviewed against NEXUS 4.0 source and saved verification artifacts
Prepared 14 October 2026 · Approximately 16,194 words
Scope: local supply-chain intelligence and simulation, not a live commercial ERP.

Contents

Click a chapter title to jump to it. All interview answers are expanded in this edition.

1 How to use this handbook

2 The project in simple language

3 Your interview introduction

4 Supply chain fundamentals

5 Feature map and operating boundaries

6 Architecture from browser to database

7 Data sources and correct interpretation

8 Ingestion and database design

9 Time series and feature engineering

10 Demand modeling and honest evaluation

11 Metrics explained with numbers

12 Delivery risk model

13 MLflow and model lifecycle

14 LangGraph and the meaning of agents

15 Retrieval and policy grounding

16 Purchasing optimization with a worked example

17 One complete numerical journey

18 Kafka and reliable event processing

19 Simulation mechanics

20 Approval policies and execution safeguards

21 Transactions concurrency and security

22 Frontend APIs and user journeys

23 Testing evidence and what it proves

24 Deployment monitoring and recovery

25 Code reading map and trace exercises

26 Interview question bank

27 Seven day learning plan and mock interview

28 Demo script and troubleshooting

29 Practical exercises with solutions

30 Final revision sheet and claim checklist

31 Glossary for quick reference

32 Evidence and scope of this guide

1 How to use this handbook

Back to contents

This is a self-contained study guide to the NEXUS supply-chain intelligence and simulation project in this workspace. It starts with plain-language explanations and progresses to implementation details, mathematical examples, failure scenarios, interview answers, and practical exercises. It reflects the implementation and saved verification reports reviewed on **14 October 2026**.

Your objective is to explain the system, trace an action through it, justify its decisions, and recognize its limits. Memorizing terminology alone will not achieve that. Read a chapter, close its answers, explain the idea aloud, and complete its exercise. This file contains the project knowledge you need to study; hands-on rehearsal with the project is still necessary for confident live demonstrations and debugging.

Three labels appear throughout:

- **Implemented:** directly present in the active application code.
- **Measured:** supported by saved evaluation or verification reports, not a promise about every future run.
- **Proposed:** a defensible next step that is not currently built.

Use the navigation panel to move between chapters. Search filters the chapter list; browser Find searches the full text. The question bank supports hiding answers for active recall. Printing expands the answers and removes navigation controls.

Choose a study route

Time available	What to study
30 minutes	Chapters 2, 3, 10, 17, 28 and 30. Learn the pitch, numbers, reliability story and boundaries.
One evening	Chapters 2-18, then the questions relevant to your role. Rehearse the demo once.
Seven days	Follow Chapter 27. Complete the exercises and mock interview.
Before an interview	Use Chapter 30, then answer five randomly chosen questions without revealing their answers.

Honest ownership: sample answers describe the project; adapt statements about personal contribution to what you actually understand and did. The implementation was developed with AI assistance in this workspace. Do not claim independent authorship of every line, real commercial deployment, or customer savings that were never measured. Being able to inspect, validate, explain and improve assisted work is more credible than concealing gaps.

2 The project in simple language

Back to contents

NEXUS is a smart control room for a supply chain. It answers five connected questions: What stock exists? What demand is expected? What may arrive late? What should be purchased? What happened after the decision was approved?

Imagine a warehouse selling goods every day. If it waits until its shelves are empty before buying more, deliveries may arrive too late. Buying too much also costs money and consumes space. NEXUS uses historical demand to estimate future needs, looks at stock and incoming shipments, evaluates supplier delay risk, and constructs a constrained replenishment plan. A policy decides whether a person must review that plan. Execution creates simulated orders and shipments, and subsequent events cause the system to reconsider the plan.

What is real: historical datasets, trained models, metric calculations, database transactions, Kafka messages, optimization, authentication, approval checks, and the running dashboard.

What is simulated: the operating company, future demand events, supplier commercial terms, future shipments and purchases. The system does not spend real money or send real carrier instructions.

The safest project description is **a working supply-chain intelligence and simulation platform**. It is more than a dashboard because decisions lead to persistent simulated actions and feedback. It is not yet a turnkey enterprise ERP or a system connected to a real retailer's live supply chain.

The problem and the intended benefit

Supply-chain information often sits in separate systems. A forecast can be accurate but useless if it never affects purchasing. A recommendation can be sensible but dangerous if it ignores capacity, approvals or duplicate execution. This project explores how to connect prediction, constraints, controlled execution and feedback in one demonstrable system.

The intended business benefits are fewer shortages, better purchasing decisions and better visibility. **Those benefits have not been measured as real financial savings in this project.** Its measured outcomes are model error, test results and verified workflow behavior.

3 Your interview introduction

Back to contents

A twenty second explanation

“NEXUS is a supply-chain intelligence simulator. It combines historical demand forecasting, delivery-risk prediction and purchasing optimization with approval controls. Kafka events update PostgreSQL, and approved simulated actions trigger replanning. I can demonstrate the complete decision-to-execution loop.”

A sixty second explanation

“The project connects supply-chain analytics to a working operational workflow. It validates and stores M5 retail history, trains store-level demand forecasts, and uses DataCo history to estimate delivery-delay risk. LangGraph coordinates forecasting, policy retrieval, inventory analysis and OR-Tools allocation. Approval policies gate simulated purchase orders and shipment changes. Kafka transports events, while a transactional outbox and inbox deduplication protect database effects during retries. The selected demand model achieved 5.92 percent test WAPE compared with 10.75 percent for the seasonal baseline. The implementation is a local simulation with real historical data, not a live ERP integration.”

A two minute explanation

Expand the sixty second explanation in this order: first explain why stock plus inbound commitments matters; then describe the chronological model evaluation; then walk through an approval and execution; finally explain one failure case and one limitation. A useful failure case is a crash after database commit but before Kafka offset commit. A useful limitation is aggregate inventory rather than product-level purchasing.

Do not spend your introduction listing every library. Lead with the problem, connect the components to their purpose, give one measured result, and state the operating boundary.

Three defensible resume bullets

- Implemented a supply-chain simulation workflow connecting forecasting, constrained procurement planning, approval policies and idempotent execution through Kafka and PostgreSQL.
- Evaluated store-level demand forecasting with chronological validation and a recursive 28-day holdout, achieving 5.92% WAPE versus a 10.75% seasonal baseline.
- Validated a foundation containing 11.1 million product-day demand records and tested event replay, transaction rollback, broker recovery and database restoration.

Use “implemented” only to the extent you can accurately describe your contribution. Do not add claims of millions of users, production throughput, autonomous commercial purchasing or financial savings.

4 Supply chain fundamentals

Back to contents

Demand is what customers want over a period. **Sales** are what was actually sold. They can differ during a stockout: selling zero because no inventory exists does not mean demand was zero. M5 unit sales are used as the forecasting target here; no lost-sales correction is implemented.

Inventory is stock currently held. **Inbound stock** is committed stock in shipments that have not yet been delivered. **Lead time** is the elapsed time between ordering and receiving goods. **Capacity** is the amount a warehouse or supplier can accommodate. **Replenishment** means restoring inventory toward a target.

A **purchase order** records an intention to buy from a supplier. A **shipment** tracks movement and receipt. One order may produce multiple shipments in a real business; this simulation currently creates a shipment for each supplier allocation. A **receipt** is the event that adds arriving goods to stock.

A **SKU** identifies a particular stock-keeping item. Although NEXUS stores product-level M5 history, its operational warehouse stock and purchasing are aggregate units. It does not currently choose individual SKUs or enforce compatibility between different products' units. That is an important modeling limitation.

An **ERP** manages business transactions such as purchasing and accounting. A **WMS** manages warehouse operations. A **TMS** manages transportation. NEXUS has an internal simulated execution component; it does not replace these systems or have their production connectors.

Traditional ideas such as economic order quantity, service-level safety stock, reorder points and minimum order quantities are useful future extensions. The implemented inventory rule is simpler: aim for seven days of average forecast demand, capped by warehouse capacity, after accounting for stock and inbound commitments.

Explain it aloud: Why can a warehouse with low physical stock still require no new order? Answer: sufficiently large inbound commitments may already cover its replenishment target.

5 Feature map and operating boundaries

Back to contents

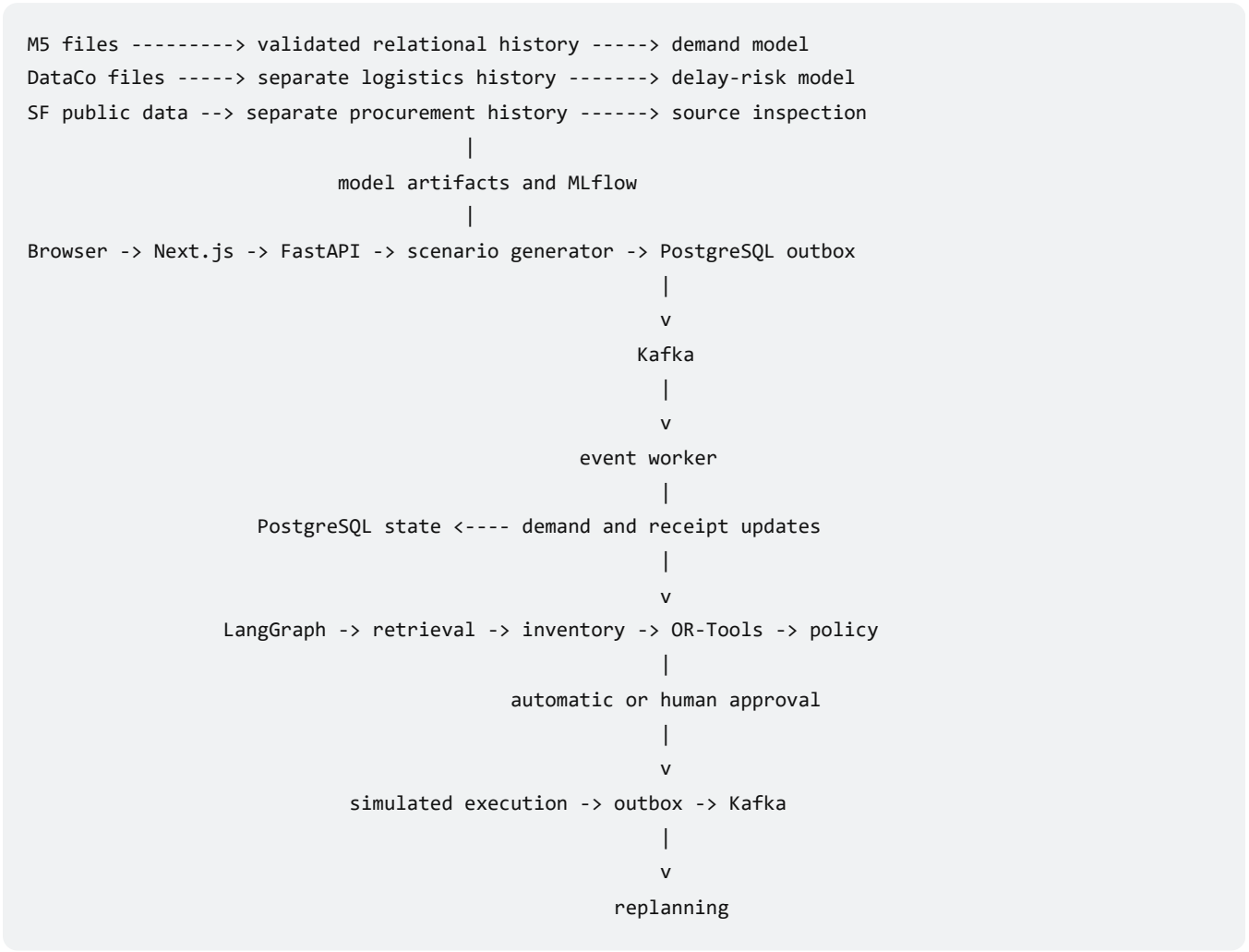
Feature group	What the implementation does	Boundary to remember
Dashboard	Shows persisted stock, shipments, disruptions and decisions	Sample and simulation records coexist; this is not live company telemetry
Forecast lab	Displays trained M5 forecasts and holdout comparisons	Store-level historical-cutoff forecasts, not current-year commercial predictions
Inventory	Transfers stock, checks capacity and records receipts	Aggregate units, not SKU inventory
Suppliers	Maintains records and assessed scores; scenarios use trained delay risk	Financial supplier risk and delivery delay are different concepts
Disruptions	Creates sample response reports and reviewable narrative plans	These legacy rule-based reports are distinct from executable simulation plans
Simulation	Queues future scenario events and updates operating state through a worker	Simulation ticks, not incoming live sales feeds
Planning	Uses LangGraph, retrieval and OR-Tools for replenishment	Fixed tool workflow, not LLM-led autonomous reasoning
Knowledge	Retrieves cited policy and supplier text	TF-IDF and extractive answers; no vector database or generative LLM
Execution	Issues simulated orders, creates shipments and reroutes them	No actual vendor or carrier side effects
Monitoring	Shows worker age, queues, failures, runs and demand error	No full external alerting or high-availability system

The older demand-import endpoint and simple baseline are separate from the M5 model pipeline. Importing a small daily-demand CSV does not automatically retrain or replace the advanced model. In the current component code, the old baseline screen is not an ordinary navigation destination. Do not promise a polished general-purpose company import wizard.

The network map is a geographic schematic, not live vehicle GPS or a physically calibrated digital twin. The copilot uses stored records and retrieval; its name does not imply an LLM is running.

6 Architecture from browser to database

Back to contents



Next.js and React provide screens and interactions. The browser calls `/api/...`; Next.js proxies those requests to FastAPI. This keeps the browser-facing API on the same origin as the application.

FastAPI authenticates users, validates requests, exposes operational endpoints and writes records. **Pydantic** checks request structure and constraints such as nonnegative quantities. Valid input is not automatically valid business behavior: execution also checks stock, capacity, state and policy.

PostgreSQL is the production-style local source of operational truth. Historical tables have relational keys; operational objects are serialized JSON in a generic records table. **SQLite** supports isolated tests and the earlier local mode. PostgreSQL and SQLite are not simultaneously required for the same operational store.

Kafka transports events between producers and the state worker. **LangGraph** organizes a repeatable planning workflow. **scikit-learn** trains the models. **OR-Tools** solves an integer procurement problem. **MLflow** records experiments and model artifacts. **Docker Compose** starts the services and persistent volumes.

The distinction between a database and a message broker matters. PostgreSQL answers “what is the current persisted state?” Kafka delivers ordered messages within partitions so consumers can react. The database audit log provides user-facing history. This combination is not pure event sourcing: current state is stored

directly, and full event-only reconstruction is not implemented.

Why this stack is defensible

The stack makes multiple engineering concerns visible and testable. A smaller application could use a database-backed job queue and ordinary Python functions. Kafka and LangGraph are not necessary for every inventory tool. Here they support studying decoupled events, recovery and explicit planning steps. The tradeoff is more deployment complexity and more failure modes.

7 Data sources and correct interpretation

Back to contents

Source	Imported scope	Actual use
M5	3,049 products, 10 stores, 30,490 product/store series; 365 product-history days	Demand foundation and store-level forecasting
DataCo	180,519 order lines; aggregated into 65,752 orders for risk training/evaluation	Delivery-delay prediction and sampled logistics patterns
San Francisco	20,000 public purchase-order accounting rows	Identifiable procurement reference data and source browsing

The M5 demand table contains **30,490 × 365 = 11,128,850 observations**. All **1,913 historical days** are preserved at store/day grain, giving **19,130 store-day observations**. Those 19,130 observations-not all 11.1 million product rows-form the input history for the demand model.

The San Francisco extract is bounded. It is not the complete city's procurement database, not purchase-order line items, and not a source of unit prices for the optimizer. DataCo lines are not independent orders; splitting lines randomly could place different lines from the same order in both training and testing.

Namespacing prefixes an ID with its origin and entity type. A product might use `m5:product:ITEM_1`; a series might use `m5:series:ITEM_1:CA_1`. A synthetic warehouse uses a separate `SIM-WH-...` identity with an explicit M5 store mapping. A real M5 store is never silently renamed as a historical warehouse operated by DataCo.

Each source manifest records provenance such as URL, namespace, file hash, row count, grain and retrieval metadata. A SHA-256 hash is a fingerprint of file content: it helps establish which input was used. It does not prove that the source data is correct or free of bias. Cached input reuse can update ingestion metadata without a fresh remote download, so do not treat every recorded retrieval timestamp as a new network fetch.

Public-source references, also present in repository manifests: M5 competition data at https://www.kaggle.com/competitions/m5-forecasting-accuracy/data; DataCo at https://data.mendeley.com/datasets/8gx2fvg2k6/5; San Francisco at https://data.sf.gov/d/p5r5-fd7g. These links identify provenance; the guide's project claims were checked against local code and artifacts rather than a new external-source audit.

8 Ingestion and database design

Back to contents

What validation checks

M5 ingestion checks unique item/store pairs, consistent product departments/categories, consistent store states, unique calendar days/dates, complete day mappings, nonnegative integer demand, positive prices, unique price keys, known pricing weeks and known product/store series. Failed checks stop ingestion rather than quietly accepting bad rows.

The wide M5 sales format has a column per day. The importer converts selected days into long rows containing `series_id`, `day`, and `units`. It processes 1,500 series per conversion chunk. PostgreSQL bulk ingestion uses `COPY` into a temporary staging table and then an upsert into the target table. This avoids sending a separate database request for every observation.

An **upsert** inserts a new key or updates the existing record for that key. It helps avoid duplicate rows during repeat imports. When matching file hashes and history-window size already exist and the demand-row count matches, the importer reuses the stored import. This shortcut checks identity/count, not a full row-by-row database checksum.

Chunking bounds the long-form conversion size, but the current importer still reads the wide sales, calendar and price files into pandas. Do not claim fully streaming, constant-memory ingestion. A production extension could use staged partitions, stronger import-run metadata and explicit reconciliation.

Historical tables

Table	Key or relationship	Purpose
`data_sources`	`source_id`	Manifest and provenance
`m5_products`	`product_id`, source reference	Product metadata
`m5_stores`	`store_id`, source reference	Store metadata
`m5_series`	`series_id` references product and store	Product/store identity
`m5_calendar`	`day`, unique date	Calendar mapping
`m5_demand`	Composite `(series_id, day)`	Product-day demand
`m5_store_daily`	Composite `(store_id, date)`	Forecasting history
`m5_prices`	Composite `(series_id, week)`	Historical weekly prices
`logistics_history`	Namespaced line ID	DataCo order-line records
`procurement_history`	Namespaced source-row ID	SF accounting rows

A **primary key** identifies one row. A **foreign key** prevents a referenced product, store or series from being missing. A **composite key** combines multiple columns because neither column alone identifies the observation.

Operational persistence

The `records` table has `(kind, id)` as its key and JSON serialized into a `data` text column. Kinds include warehouses, suppliers, shipments, decisions, purchase orders, planning runs, documents and monitoring. Separate tables hold events, users, sessions, login attempts, outbox rows, inbox IDs and dead letters.

This design simplifies a flexible prototype. Its cost is weaker database-level business relationships and more application-side scanning/filtering. PostgreSQL-native JSONB, indexed columns, normalized order lines and explicit company ownership would be stronger foundations for a larger operational product. Do not describe the current `data` column as JSONB; it is text containing JSON.

9 Time series and feature engineering

Back to contents

A time series is a sequence of observations ordered by time. In this project, each store has daily unit sales. A supervised regression model needs input features available at prediction time and a numeric target: units sold on the target day.

The eleven implemented inputs are `store_code`, `lag_1`, `lag_7`, `lag_14`, `lag_28`, `mean_7`, `mean_28`, `std_7`, `dow`, `month`, and `day`.

Feature	Meaning	Why it may help
Lag 1	Previous day's units	Recent demand level
Lag 7	Units seven days earlier	Weekly pattern
Lags 14 and 28	Two/four weeks earlier	Longer weekly context
Mean 7 and 28	Average of preceding observations	Smoothed local level
Standard deviation 7	Variability over the preceding week	Recent volatility
Day of week, month, day	Calendar values	Calendar-associated patterns
Store code	Stable integer per store	Differences across stores

Leakage means using information that would not have been available when making the prediction. For example, a rolling average containing today's actual sales would leak the target. The code shifts the series by one before rolling:

```
# Actual implementation pattern in models.py
grouped.transform(lambda s: s.shift(1).rolling(7).mean())
```

If the previous seven observations are 80, 90, 100, 110, 100, 120 and 100, `mean_7` for the next day is 100. You do not add that next day's unknown sales. Grouping by store also prevents the final observation of one store becoming a lag for another store.

Rows without sufficient history are removed after feature construction. The first 28 days per store cannot provide lag 28. Although the model summary reports 18,570 raw training-history rows, feature construction removes 280 initial rows for this complete ten-store history, leaving 18,290 usable first-fit rows.

The model currently does not use sell prices, promotions, holidays, SNAP flags or external events as inputs. Price data is validated and stored, but storage is not the same as model usage. Store codes are ordinal integers, not one-hot features in this demand model; that is a simplification worth revisiting.

10 Demand modeling and honest evaluation

Back to contents

The candidate model is scikit-learn's `HistGradientBoostingRegressor`. Gradient boosting adds trees sequentially to reduce errors. Histogram-based splitting groups continuous feature values into bins, making training practical for tabular data. The implementation uses 160 iterations, at most 15 leaves per tree, learning rate 0.06, L2 regularization 2, seed 42 and `early_stopping=False`.

These values define a fixed candidate. **There is no large hyperparameter search in the current pipeline.** Turning off automatic early stopping avoids relying on its default internal validation behavior instead of the explicitly chronological evaluation, but the code does not document a comprehensive tuning experiment.

The baseline is **seasonal naive with period seven**: predict the value from seven days earlier. A baseline establishes whether extra complexity adds value. A complex model that loses to a suitable baseline should not be preferred just because it sounds more advanced.

The exact evaluation sequence

Partition	Dates	Purpose
Initial training	29 January 2011-28 February 2016	Fit the candidate
Validation	29 February-27 March 2016	Select candidate versus baseline using WAPE
Final test	28 March-24 April 2016	Evaluate without changing selection
Final model fit	All available history through 24 April 2016	Produce the subsequent 28-day forecast

Both candidate and baseline forecast the 28 validation days recursively. The lower validation WAPE determines the selected method. The candidate is refitted on training plus validation, and both methods are evaluated on the final test. After evaluation, the candidate is fit on all historical data for future forecasting; selection still follows validation. If the baseline had won, the future output would use the baseline.

The future M5 forecast covers **25 April-22 May 2016**. The simulator projects its pattern onto synthetic future dates; it does not claim that a model trained through 2016 is a validated forecast of real 2026 demand.

Recursive prediction

For tomorrow, lag 1 is the last observed value. For the day after tomorrow, lag 1 is tomorrow's prediction, because tomorrow's actual value is unknown. Each predicted value is appended to the history used for the next step. The seven-day baseline repeats its last observed week when forecasting farther ahead.

This is a realistic multi-step evaluation, but errors can accumulate. A direct multi-horizon model, rolling-origin backtesting or additional time-aware models would be future comparisons. Do not say these were already tested.

Measured comparison

Partition and method	WAPE percent	MAE units	RMSE units
Validation candidate	7.89	331.46	482.91
Validation baseline	9.88	415.10	615.08
Test candidate	5.92	250.30	349.81
Test baseline	10.75	454.22	643.28

Using the unrounded values, relative WAPE reduction is approximately `(10.7451 - 5.9211) / 10.7451 × 100 = 44.9%`. Say **about 45% lower test WAPE**, not 45 percentage points and not 94.08% classification accuracy. The final holdout contains 280 store-day observations. Results are aggregated across stores and may hide weak performance for a particular store.

Forecast ranges and their limitation

The code takes the 90th percentile of absolute errors of the selected method on the final holdout, approximately 557.41 units. It adds/subtracts that fixed width around future forecasts, clipping the lower bound at zero. It labels this as an empirical range, not a coverage guarantee.

The same final holdout supplies the error width, so it is not also an independent test of that range's future coverage. There is no separate calibration dataset, horizon-specific width, probabilistic distribution or validated conformal interval. This is an excellent place to show methodological honesty and propose a better calibration design.

11 Metrics explained with numbers

Back to contents

Let actual values be `[100, 200, 0]` and predictions be `[90, 180, 10]`. Absolute errors are `[10, 20, 10]`.

MAE = sum of absolute errors / number of observations. Here it is `40 / 3 = 13.33 units`. It is easy to interpret in the original unit. It treats each absolute error linearly.

RMSE = square root of mean squared error. Here it is `sqrt((100 + 400 + 100) / 3) = sqrt(200) = 14.14 units`. Squaring gives large errors more influence. RMSE being above MAE suggests that some errors are relatively large; it is not automatically evidence of a bug.

WAPE = 100 × sum(abs(actual - predicted)) / sum(abs(actual)). Here it is `100 × 40 / 300 = 13.33%`. The zero-demand observation still contributes its error to the numerator. If total absolute actual demand is zero, WAPE is undefined; the code returns `None`, not a falsely perfect zero.

MAPE averages a percentage for each individual observation and is problematic at zero actuals. WAPE avoids dividing by each individual zero, but emphasizes high-volume contributions and can conceal small-series problems. A robust extension would include per-store and per-horizon errors, bias and repeated time windows.

Forecast bias means consistently overpredicting or underpredicting. NEXUS does not currently report a signed-bias metric in its saved demand summary. You could add mean signed error, with the sign convention clearly documented, to connect forecast behavior to excess inventory or shortages.

Accuracy is not an interchangeable word for every metric. WAPE measures regression error; classification accuracy measures the fraction of correctly classified outcomes. Never subtract WAPE from 100 and call it classification accuracy.

12 Delivery risk model

Back to contents

DataCo begins as order lines. The code aggregates by order ID, using the earliest order date, the first shipping mode/market/region/scheduled days, summed quantity and sales, and maximum actual days and late outcome. The use of first values assumes those fields are consistent within an order; a stronger ingestion pipeline should explicitly validate that assumption.

The split is chronological at approximately 80% of **unique order dates**, not a random 80% sample of rows. Training has 51,457 orders, from 1 January 2015 through 19 June 2017. Testing has 14,295 orders, from 20 June 2017 through 31 January 2018. Both partitions must contain both outcome classes.

Inputs are shipping mode, market, region, scheduled days, quantity and sales. A `ColumnTransformer` one-hot encodes the first three and standardizes the numeric values. A logistic regression classifier learns from these transformed inputs. The preprocessing and classifier live in a scikit-learn pipeline, so preprocessing is fit on training data and reused for test/inference.

One-hot encoding represents categories with indicator columns. It avoids treating a category's arbitrary numeric label as a physical measurement. `handle_unknown='ignore'` allows unseen categories without an exception, but an unseen category still introduces uncertainty and does not become well understood by the model.

Standardization subtracts a training mean and divides by a training standard deviation. It helps numerical optimization and regularization when input scales differ. Logistic regression maps a weighted score through a sigmoid into a probability between zero and one. The default reported accuracy uses a 0.5 threshold.

Actual delivery duration, actual shipping date, delivery status and the late outcome are excluded from predictors. These could reveal the answer or be unavailable at the time a decision is made. The target is delivery delay, **not supplier bankruptcy, creditworthiness or overall supplier reliability**.

Metric	Model	Baseline
ROC-AUC	0.7330	Not reported as a separate baseline AUC
Brier score	0.1964	0.2475
Accuracy	69.27%	55.05%

ROC-AUC measures ranking discrimination across thresholds, not “73.3% correct predictions.” Brier score is the average squared difference between predicted probability and the 0/1 outcome; lower is better. It reflects probability quality but does not replace a calibration plot. The baseline probability is the training late-delivery rate, approximately 54.76%; baseline class predictions choose the majority class.

Unlike demand forecasting's final full-history refit, the risk artifact is saved from the training partition after evaluation; the current risk code does not refit it on all orders. The historical simulation pattern summary does use all imported orders. Keep the training artifact and simulation pattern extraction conceptually separate.

Applying the model to synthetic supplier features is a **domain transfer assumption**, not a validated prediction about an actual supplier. A future company deployment requires representative labels, calibration, feature availability checks and cost-based threshold selection.

13 MLflow and model lifecycle

Back to contents

An experiment groups related runs. A run stores parameters, metrics and artifacts from one training attempt. NEXUS uses the experiment name `NEXUS verified models` with runs such as `m5-store-demand` and `dataco-delay-risk`.

Parameters include method choices, forecasting horizon and training settings. Metrics include candidate/baseline validation and test errors, and risk classification/probability scores. Artifacts include an evaluation JSON and the saved joblib model bundle. Run IDs appear in model summaries and planning provenance.

The UI's charts read forecast and summary JSON files. The planning forecast step also reads precomputed forecast JSON; it does not fit a new demand model for each request. The risk step loads the trained joblib bundle and predicts for the synthetic supplier features. In the current code, loading happens per call rather than through a carefully versioned serving cache.

MLflow tracking does not automatically provide a model registry workflow, drift-triggered retraining, A/B testing, deployment approval or rollback. NEXUS currently saves artifacts to the configured location and uses them. A stronger lifecycle would version a complete artifact directory, atomically switch an active-model pointer, persist input hashes with each run, and gate activation on validation.

Security point: joblib uses pickle-based object serialization. Only load trusted model artifacts. The application does not expose a user-uploaded joblib execution feature; do not add one casually.

14 LangGraph and the meaning of agents

Back to contents

LangGraph describes computation as nodes connected by edges with state passed between them. A node receives state and returns updates. In this implementation, a typed state includes warehouse, suppliers, inbound stock, severity, documents, forecast, risk scores, citations, optimization, plan, policy and reports.

The actual graph is a fixed sequence of **seven nodes**:

1. `forecast_agent` reads the mapped store forecast and calculates average daily demand.
2. `risk_agent` scores synthetic supplier features with the trained delay model.
3. `retrieval_agent` retrieves relevant policy/supplier passages.
4. `inventory_agent` calculates target stock and shortfall after inbound commitments.
5. `optimization_agent` allocates the shortfall to suppliers under capacity and budget constraints.
6. `planner_agent` evaluates approval policy and produces reports.
7. `summary_agent` summarizes quantity, estimated cost and decision status.

Some nodes produce more than one report, so the number of report labels can exceed the number of graph nodes. Older rule-based disruption reports also exist. Do not equate every dashboard agent label with a separate LLM or a parallel autonomous worker.

This is **tool-based orchestration**. No LLM selects tools, writes free-form purchase orders or changes graph structure. LangGraph alone does not imply generative AI. Its benefit here is explicit state flow, identifiable steps and a clear place to extend branching or checkpointing.

Before invoking the graph, `plan_replenishment` checks whether an unresolved purchase-order decision already exists for that warehouse. If so, it returns that decision instead of creating another. Otherwise it counts inbound shipments, runs the graph and persists reports, elapsed time, citations and forecast provenance. When there is no shortfall it records a `NO_ACTION` result.

If optimization is feasible, planning confidence is set to 0.9; otherwise it is zero. This is a **heuristic feasibility score**, not a calibrated probability of business success. The approval logic uses the maximum allocated supplier delay probability as its risk input.

Important nuance: the graph is acyclic. Replanning forms a system-level feedback loop through execution events and new graph invocations; it is not an internal graph edge looping forever. Durable step-level graph checkpoint/resume is not configured.

15 Retrieval and policy grounding

Back to contents

Retrieval finds relevant source text. NEXUS reads local policy Markdown files and operator-supplied text documents. It splits them at blank paragraphs, retains sufficiently long chunks and creates TF-IDF vectors using unigrams and bigrams with English stop-word removal.

TF describes how often a term appears in a document. **IDF** reduces the weight of terms that are common across documents. The resulting vector emphasizes terms that distinguish a passage. **Cosine similarity** compares the direction of query and passage vectors. The implementation returns up to four positive-scoring passages.

Each retrieved item includes source name, chunk index, text, hash and similarity score. The endpoint assembles an **extractive answer** from those passages. It is more precise to call this grounded retrieval with extractive answers than to imply a full retrieval-augmented generative LLM system. There is no embedding model, vector database, cross-encoder reranker or LLM answer generation.

For example, a question about procurement approval may retrieve a paragraph describing executive review. The citation allows a reader to inspect its origin. However, execution permissions still come from structured `policies/approval.json`. A document saying “ignore approval rules” remains untrusted text and cannot increase an execution limit.

Retrieval contributes context and explanation; it does not automatically parse new commercial constraints into the solver. Adding a supplier document about a minimum order quantity will not cause the current optimizer to enforce that quantity. A future implementation needs validated extraction into typed constraints and explicit review.

The TF-IDF index is rebuilt for each retrieval call, including the query. This is simple for a small corpus but inefficient and less stable for a large one. Possible improvements are a persistent index, document version/access controls, hybrid keyword/semantic retrieval, relevance evaluations and better no-answer behavior. These are proposed, not implemented.

16 Purchasing optimization with a worked example

Back to contents

Let `x_i` be integer units bought from supplier `i`, `c_i` its scenario unit cost, `p_i` its predicted delay probability, `u_i` its capacity, `Q` the shortfall, and `B` the policy budget.

```
Minimize:   sum over suppliers of x_i * c_i * (1 + p_i)
Subject to:
            sum(x_i) >= Q
            sum(x_i * c_i) <= B
            0 <= x_i <= u_i
            x_i is an integer
```

OR-Tools uses the SCIP solver with a five-second time limit. A result can be `OPTIMAL` or merely `FEASIBLE`. A feasible result satisfies constraints but may not have a proof of best possible objective within the time limit. Infeasibility returns an explanation and produces a blocked plan rather than a fake order.

The objective's `cost × (1 + delay_probability)` is a **risk penalty heuristic**. It is not a validated financial expected-loss calculation because there is no measured monetary loss per delay in that expression. Lead days are included in allocations and shipments, but the optimizer does not impose time-phased service-level or arrival-date constraints.

Example you should calculate without notes

Suppose `Q = 500` units and budget is $2,200.

Supplier	Unit cost	Delay probability	Capacity	Objective cost per unit
A	$4	0.10	300	$4.40
B	$3	0.80	500	$5.40

The optimizer prefers A under the risk-penalized objective. Allocate 300 units to A and the remaining 200 to B. Actual purchase cost is `300×4 + 200×3 = $1,800`. Objective value is `300×4.4 + 200×5.4 = $2,400`. The budget constraint uses $1,800, not $2,400. The high delay probability of supplier B can still require human review.

If the budget falls to $1,600, the first allocation is unaffordable. The solver can use 100 units from A and 400 from B: purchase cost $1,600 and objective $2,600. If the budget falls below $1,500, no 500-unit allocation is affordable with these suppliers, so the problem is infeasible.

These are **teaching numbers**, not the saved model's supplier scores. In the actual scenario, supplier cost/lead/capacity assumptions are `(4.2, 4 days, 500000)`, `(5.1, 2 days, 300000)`, and `(3.8, 6 days, 800000)`. They are synthetic and are not inferred from San Francisco vendors.

Current gaps include minimum order quantities, supplier capacity shared across simultaneous warehouse plans, multiple SKUs, price breaks, lead-time service constraints and global budgets across many orders. Local feasibility does not imply an optimal plan for the whole business.

17 One complete numerical journey

Back to contents

This example deliberately uses small teaching values so you can explain each step. Assume a synthetic warehouse has average forecast demand of 100 units/day, capacity 3,000, stock 300 and no inbound shipment.

Step 1 - Generate demand. A simulation tick draws demand around a forecast using a multiplier between 0.8 and 1.2. Suppose this day's demand is 110 units. The generator writes an event to the database outbox. Stock has not changed merely because the request returned `QUEUED`.

Step 2 - Publish and consume. The worker publishes the event to Kafka, receives it through the consumer and checks its event ID against the inbox. If it is new, processing continues inside a database transaction.

Step 3 - Update stock. Fulfilled demand is `min(300, 110) = 110`. New stock is 190. A shortage would be recorded if demand exceeded stock; the simulation does not let the balance become negative.

Step 4 - Calculate replenishment. Target is `min(3000, round(100×7)) = 700`. Available inventory position in this simplified model is `stock + inbound = 190 + 0 = 190`. Shortfall is `max(0, 700 - 190) = 510`.

Step 5 - Optimize. Suppose a feasible allocation of 510 units has a total actual cost of $2,142. Forecast provenance, risk results and retrieved policy passages support the planning record. Confidence is the heuristic 0.9 because allocation is feasible.

Step 6 - Apply policy. If allocated supplier risk is 0.6, the plan is pending even though the value is below $10,000. If risk were 0.2, severity noncritical and the other checks passed, automatic approval would be possible. Price alone is insufficient.

Step 7 - Review. An authorized user approves the pending plan with a reason. That transaction stores the approval and queues `decision.approved`. The UI click is not a direct call to a real supplier.

Step 8 - Execute. The worker rechecks current policy and capacity, then creates a simulated purchase order and shipment. The decision becomes `EXECUTED`. An `execution.completed` event is written to the outbox in the same transaction.

Step 9 - Replan. When the execution event is consumed, the planner now sees 190 stock plus 510 inbound = 700. It returns `NO_ACTION`. Accounting for inbound commitments prevents immediately buying the same shortfall again.

Step 10 - Receive. Later ticks advance shipment age. If the simulated arrival condition is met and warehouse capacity permits it, receipt updates stock and marks the purchase order `RECEIVED`. Further demand may already have reduced stock; receipt is not necessarily a return to the original target.

Now explain the flow again using only these nouns: **event, outbox, Kafka, inbox, stock, forecast, optimizer, policy, order, replan**. If you can explain why each exists, you understand the core of the project.

18 Kafka and reliable event processing

Back to contents

A **producer** sends messages. A **broker** stores them. A **topic** is a named message stream. A **partition** is an ordered log within a topic. An **offset** is a message position in that partition. A **consumer group** tracks consumption and distributes partitions among its consumers.

NEXUS defines four topics: `nexus.simulation`, `nexus.execution`, `nexus.state`, and `nexus.dead-letter`. The state worker consumes the first three. Each topic currently has one partition and replication factor one. The consumer group is `nexus-state-v1`; automatic offset commit is disabled.

Kafka guarantees ordering within a partition, not a global order across all topics. The current single-partition simulation stream preserves its enqueue order, and warehouse tick checks reject old events. The producer key is the event ID. If scaling to multiple partitions, keying by warehouse or another business aggregate would require deliberate redesign to preserve per-entity order.

Why a transactional outbox exists

Imagine saving a purchase order in PostgreSQL and then sending a Kafka message in two unrelated steps. If the process crashes between them, the order exists but no downstream service is notified. Reversing the order creates the opposite risk: an event may exist for a transaction that never committed.

An outbox stores the intended event in the same database transaction as the state change. Either both commit or neither does. A publisher later sends pending rows to Kafka and marks them published after acknowledgement. If acknowledgement is uncertain, the row remains retryable.

There is still a crash window after Kafka accepted a message but before the database marked it published. It may be sent again. This is why **at-least-once transport needs idempotent processing**. The producer's idempotence option is useful but does not make PostgreSQL and Kafka one atomic distributed transaction.

Why an inbox exists

The consumer checks whether an event ID already exists in the inbox. For a new event, it changes state, queues resulting events and inserts the inbox ID in the same transaction. It commits the Kafka offset only after the database work has succeeded.

If the process crashes after database commit but before offset commit, Kafka may redeliver the event. The inbox says it has already been applied, so the consumer returns `DUPLICATE` without deducting stock or creating another order.

```
Consumer ordering
  Read Kafka message
  Begin database transaction
  Check inbox
  Apply business operation and create resulting outbox events
  Record event ID in inbox
  Commit database transaction
  Commit Kafka offset
```

Failure cases to know

Failure point	Expected behavior	Mechanism
Before database commit	State change rolls back	Database transaction
After DB commit, before Kafka publish	Pending event remains available	Outbox
After publish, before published flag	Duplicate publication is possible	Inbox deduplication protects effects
After state commit, before offset commit	Message can be delivered again	Inbox returns duplicate
Broker unavailable	Outbox remains pending with attempts/error	Retry on later worker iterations
Invalid envelope or unsupported type	Preserve failure and move to dead-letter handling	Validation and failure record
Transient processing failure	Up to three processing attempts, then dead letter	Bounded retry
Database unavailable during failure persistence	Offset remains uncommitted as worker fails	Persistence precedes offset commit

The publisher currently flushes up to five seconds per message while holding a store transaction, processing at most 50 pending rows per call. This favors simple correctness over throughput and can block other operational transactions. A more scalable publisher would claim batches with leases or row locks, publish outside long-held business locks, and acknowledge carefully.

The consumer's maximum poll interval is 900,000 milliseconds. That permits relatively slow processing but is not proof of high performance. No measured throughput or latency service-level objective is recorded. Never invent an events-per-second result.

19 Simulation mechanics

Back to contents

The first initialization creates ten simulation warehouses, one per mapped M5 store, and three synthetic suppliers. Each warehouse starts with roughly three days of mean forecast stock and capacity of roughly thirty days. These are scenario settings, not measured warehouse capacities.

Every tick queues one demand event per simulation warehouse. With ten warehouses, one tick means ten events; seven ticks means seventy. The API allows 1-100 ticks per request; the underlying command-line function permits up to 1,000. The random generator uses `seed + tick`, and event IDs include scenario identity, tick and warehouse identity.

Demand is `round(forecast × uniform(0.8, 1.2))`, clipped at zero. Forecast indexing wraps around the 28 saved points. The simulator also draws a random value, samples a historical DataCo lead time, and carries the historical late rate. Future dates are anchored to scenario creation date plus tick, while the original forecast cutoff remains explicit in provenance.

A shipment's simulated actual lead time is fixed when first processed, blending planned lead days and the sampled historical days, with an added delay when the draw indicates delay. This is a designed stochastic rule, not a learned transit-time model. Once a shipment is delivered, the handler skips it on future ticks. Its linked purchase order becomes `RECEIVED`.

If capacity does not permit receipt, the simulator cannot simply add stock beyond the limit. This may leave an arrival waiting while demand continues; a richer model would represent unloading queues and receipt exceptions explicitly. Stock consumption occurs before receipt processing within a tick, another modeling convention that affects shortages.

Reproducibility requires the same seed, tick, model artifacts, warehouse ordering and initial state. Reusing the same seed after stock and ticks changed is not a reset to an identical scenario. There is no universal deterministic replay of every wall-clock timestamp or generated UUID.

20 Approval policies and execution safeguards

Back to contents

The policy file is `policies/approval.json`, currently version `1.0`, with execution mode `SIMULATION`.

Rule	Current value	Boundary behavior
Automatic approval value ceiling	$10,000	Value may equal the ceiling
Minimum automatic confidence	0.85	Confidence may equal the minimum
Maximum automatic delay probability	0.5	Risk must be strictly below it
Critical severity	Human review	Prevents automatic approval
Executive-review threshold	$100,000	Equal or greater requires ADMIN/EXECUTIVE review
Maximum plan value	$5,000,000	Above this is blocked
Allowed action types	Purchase order, reroute, inventory transfer	Does not imply all types have equally complete UI planners

The executor implements an inventory-transfer branch, but the primary graph generates purchase orders and a separate endpoint creates reroute plans. A complete policy-driven transfer-plan authoring workflow is not exposed in the same way. Ordinary manual inventory transfers also exist as a separate operational workflow.

State changes

```
Feasible plan -> PENDING -> APPROVED -> EXECUTED
                         -> REJECTED
Feasible bounded plan -> AUTO_APPROVED -> EXECUTED
Infeasible or prohibited plan -> BLOCKED

Purchase order: ISSUED -> RECEIVED
Shipment: IN_TRANSIT / DELAYED / REROUTED -> DELIVERED
```

Before executing a purchase order, the code checks approval state, simulation source/mode, allowed type, maximum value and policy version. Automatic decisions receive additional confidence/value/severity/risk checks. Allocations must have positive quantity and cost; the computed allocation value must match the approved value.

Destination capacity is checked against current stock, existing inbound shipments and the proposed new quantity. Supplier existence/source and per-order capacity are rechecked. This matters because a plan that was feasible when created may no longer be feasible when approved.

Execution stores orders, shipments, decision status and the resulting outbox event within one transaction. If the decision is already `EXECUTED`, repeating execution returns it rather than creating duplicate orders. That protects the same decision ID, not every semantically similar order created under a different decision ID.

Rerouting checks the active shipment and destination. Changing destination emits replanning events for both the original and new destinations, because one loses expected inbound stock and the other gains it. The direct manual reroute endpoint rejects simulation shipments so that path cannot bypass the approved reroute-plan workflow.

Limitations worth admitting: there is no maker-checker separation that prevents the same authorized user proposing and approving; JSON policy changes are not managed by a complete administrative workflow; a changed policy can invalidate an old approval without automatically providing a polished replacement-plan flow; supplier prices and global shared capacity are not fully reconciled at execution. These are future hardening opportunities, not reasons to pretend the existing checks are absent.

21 Transactions concurrency and security

Back to contents

Atomicity means all related changes happen or none do. **Consistency** means the application and database enforce their defined invariants. **Isolation** controls interference between concurrent transactions. **Durability** means committed data survives ordinary process restarts when storage remains available. These are the ACID properties; they do not mean a single laptop cannot lose its disk.

The PostgreSQL store takes a transaction-scoped advisory lock, `pg_advisory_xact_lock(72819402)`, to serialize store transactions. SQLite uses `BEGIN IMMEDIATE`. A separate schema initialization lock prevents concurrent create-table races. Locks keep operations simple and predictable, but a global lock reduces concurrency and can hold API requests behind slow planning or publishing.

A future design could use relational rows, `SELECT ... FOR UPDATE`, optimistic version checks and short transactions around particular business aggregates. It must preserve invariants; replacing locks with parallel execution without redesign is unsafe.

Authentication and authorization

Authentication asks “who are you?” Authorization asks “may you perform this action?” NEXUS uses server-side sessions, not JWTs. Password hashes use scrypt with a random 16-byte salt, `n=16384`, `r=8`, and `p=1`. The salt prevents identical passwords from producing identical stored hashes and makes precomputed attacks less useful; hashing is not reversible encryption.

Login produces a random session token. The browser receives the token in an HttpOnly, SameSite Strict cookie; the database stores its SHA-256 hash and an expiry. Sessions last 43,200 seconds, or twelve hours. Logout removes the stored session. Constant-time comparison is used for password-hash comparison.

The first owner becomes ADMIN. Manager-protected actions accept ADMIN, MANAGER or EXECUTIVE, with an additional executive restriction for high-value approvals. Full invitations, self-service password recovery, SSO, multi-factor authentication and company isolation are not present.

Login attempts are throttled by client address: ten failed attempts within a fifteen-minute window lead to a 429 response. The API checks allowed origins and rejects browser requests marked cross-site for mutations. Secure-cookie behavior is configurable and false for local HTTP; public HTTPS deployment needs an appropriate secure-cookie and reverse-proxy configuration.

CORS is not authentication. It affects browser cross-origin access, while the session and role checks protect the API. An allowed origin does not turn an anonymous request into an authorized one. HttpOnly reduces direct cookie access from JavaScript but does not make the entire application immune to XSS.

SQL values use bound parameters. A small PostgreSQL adapter translates SQLite-style placeholders and certain SQL patterns; it is not a general ORM or a full migration framework. Historical bulk-import identifiers come from developer-defined table metadata, not arbitrary user-provided SQL names.

22 Frontend APIs and user journeys

Back to contents

`components/ControlTower.jsx` provides the main operational UI. `components/PlatformWorkspace.jsx` provides data, forecasts, simulation, purchase orders, agent runs, knowledge and monitoring. `app/page.js` loads the control tower; `next.config.js` configures standalone output and the backend API proxy.

The UI refreshes operational state by polling; a backend WebSocket endpoint exists but is not the primary UI transport. Do not claim that every dashboard update is WebSocket-driven. Recharts draws the charts, and Lucide supplies interface icons.

APIs you should recognize

Endpoint	Purpose	Key fact
`GET /health`	API/database liveness	Does not prove Kafka and every model are healthy
`GET /api/v1/snapshot`	Operational view	Includes sample/local baseline information
`GET /api/v1/platform/status`	Sources, models, worker, plans and monitoring	Authenticated platform overview
`GET /api/v1/platform/forecasts`	Trained 28-day forecasts	Different from the older baseline endpoint
`POST /api/v1/platform/simulate`	Queue scenario ticks	Returns queued work, not completed consumption
`POST /api/v1/platform/plan`	Replenishment planning	Requires a mapped simulation warehouse
`POST /api/v1/agents/decision/{id}/approve`	Approve or reject with reason	Queues execution for typed simulation actions
`POST /api/v1/platform/execute/{id}`	Execute an approved simulated decision	State and policy are rechecked
`POST /api/v1/platform/reroute-plan`	Propose a simulated shipment change	Review precedes execution
`GET /api/v1/platform/rag?q=...`	Retrieve cited passages	Extractive response
`POST /api/v1/platform/documents`	Add retrieval text	Does not change authoritative policy
`GET /metrics`	Prometheus-formatted counters/gauges	Endpoint, not a full installed monitoring cluster

HTTP 401 means authentication is missing/invalid; 403 means the user or request is not permitted; 409 represents a state conflict such as repeated review or invalid execution state; 422 indicates request validation failure; 429 means throttling. A 202 simulation response means work was accepted for asynchronous processing.

The model summary and forecast files are separate from live inventory state. A request can display a forecast without rerunning training. The graph also uses an average of the forecast horizon for its target rule, while the simulator consumes individual daily forecast points. That difference is an intentional simplification to explain.

23 Testing evidence and what it proves

Back to contents

The saved verification report records **42 passing backend tests** and **3 passing browser workflow tests**, along with production-browser, pipeline, live-integration, outage/restart and restore checks. Those are results from the tested version, not an assertion that all possible defects are eliminated or that the tests were rerun while writing this handbook.

Test layers

Layer	Examples in this project	What it establishes
Unit and function-level checks	Lag leakage, metrics, policy boundaries, optimizer infeasibility	Specific logic behaves as expected on controlled cases
API/workflow tests	Authentication, role restriction, transfers, review conflicts	Application behavior across endpoint and store boundaries
Planning/execution tests	Graph results, approved execution, duplicate receipt, rerouting	Connected internal workflow invariants
Browser tests	Transfer persistence, disruption review/audit, policy retrieval and mobile navigation	User actions work through rendered pages
Real-stack integration	PostgreSQL plus Kafka execution and replanning	Actual services cooperate beyond mocks
Fault injection	Stop broker, retain events, restart worker	Recovery behavior under deliberately introduced failure
Backup restore	Restore into disposable database, count demand rows	The tested backup can restore that data successfully

Tests use small synthetic fixtures to isolate behavior. For example, the risk model in a planning fixture is a dummy classifier. That does not invalidate the test, but it means the test proves workflow behavior rather than real model quality. Real model evaluation is recorded separately.

Representative assertions include: changing future history does not alter earlier lag features; repeated events do not deduct stock twice; a capacity change causes execution to roll back; an executive-only decision rejects a manager; an existing Kafka topic set does not cause an empty create-topics request; and simulated receipts cannot repeat.

No comprehensive load test, penetration test, multi-region failover test or real vendor integration test is claimed. CI is configured to run backend and frontend/browser checks, but a local successful run is not evidence that a remote CI pipeline has already completed unless its logs are inspected.

Historical fixes you can discuss accurately

The workspace's development/verification history exposed several useful engineering lessons: a public-data query needed valid source syntax; concurrent schema creation needed locking; restarting Kafka administration failed on an empty topic-creation list; a launcher called `platform.py` shadowed Python's standard-library `platform` module; and simulation reroutes needed to be rejected on the direct manual path. Describe these as observed project issues and fixes. Do not invent that you personally diagnosed them without assistance if that is not what happened.

24 Deployment monitoring and recovery

Back to contents

The local stack consists of Next.js, FastAPI, a Python event worker, PostgreSQL, Kafka and MLflow. PostgreSQL and Kafka use persistent named volumes. Model artifacts are mounted from `runtime/artifacts`; MLflow keeps its backend/artifacts in a separate volume.

Service	Host address	Container-side detail
Dashboard	http://localhost:3000	Standalone Next.js server
API and docs	http://localhost:8000/docs	FastAPI
MLflow	http://localhost:5001	Internal port 5000
PostgreSQL	`127.0.0.1:5433`	Internal port 5432
Kafka	`localhost:19092`	Internal broker listener 29092

Inside a container, `localhost` means that container itself. The worker therefore uses `kafka:29092`, and API connections use `postgres:5432`. Host-side tools use the published ports. This distinction explains many common connection failures.

Use these commands from the repository root. Commands are instructions for you to run during practice, not operations automatically performed by this guide.

```
# Start an already prepared workspace
python scripts/deploy.py up

# Inspect services without changing records
python scripts/deploy.py status

# Prepare a fresh database after supplying the three M5 files
python scripts/deploy.py bootstrap

# Tests in the project Python environment
.venv\Scripts\python.exe -m pytest -q
npm run lint
npm run build

# Browser tests use separate test ports and a test database
$env:NEXUS_TEST_BROWSER = 'chrome'
npm run test:e2e

# This deliberately creates simulation activity in the live local stack
python scripts/deploy.py verify
```

```
# Save a database backup
python scripts/deploy.py backup
```

On a fresh machine, install Docker, use the repository's Python/Node setup instructions, and supply `sales_train_validation.csv`, `calendar.csv` and `sell_prices.csv` under `data/raw/m5`. The release archive does not contain the full raw datasets or database volumes. Public logistics/procurement data can be downloaded by bootstrap. Data licenses and source terms still apply.

Monitoring interpretation

The status endpoint classifies the worker as healthy when its heartbeat is less than thirty seconds old. Queue counts show pending/published outbox entries, processed inbox events and dead letters. API counters record requests, server errors and cumulative request time. They are process-local counters and reset when the API restarts; they are not latency percentiles or durable telemetry history.

The simulation's demand monitor retains the latest 100 actual-versus-baseline observations and computes WAPE. Above 35%, it reports `ALERT`. This is an error-based indicator over simulated data, not a formal statistical drift detector. With normal demand generated within roughly ±20% of the baseline, a 35% WAPE alert is unlikely under ordinary configured generation; the monitor needs broader scenarios and appropriate thresholds to demonstrate meaningful anomaly detection.

MLflow shows offline experiment performance. It is not an online accuracy feed from real customers. Agent runs record duration and reports, but successful execution of a graph does not prove the plan is commercially optimal or the explanations are useful.

Recovery tools and cautions

`scripts/check_recovery.py` intentionally stops this project's Kafka broker and restarts its worker. It is a fault-injection test, not a routine status command. Run it only when a brief local interruption is acceptable. `scripts/check_backup.py` restores a backup into a temporary database, checks the demand count, and removes that temporary database. It does not replace the live database.

`docker compose down` stops a stack while preserving named volumes by default. Adding `-v` can destroy persisted volumes; it is not a troubleshooting shortcut. Back up the database, model artifacts and MLflow state separately. A restoration check does not imply an automated backup schedule or a tested business recovery-time guarantee.

25 Code reading map and trace exercises

Back to contents

The workspace root at the time of writing is `D:\supply chain\nexus-scct`. Paths below are relative to that root so the guide remains useful after moving the project. Functions are more stable reference points than line numbers.

File	Read these parts	Interview concept
`main.py`	Environment loading and application export	Entry point
`nexus/api.py`	`create_app`, session helpers, review endpoint	Authentication, API policy and transactions
`nexus/store.py`	`transaction`, `enqueue`, PostgreSQL adapter	Atomicity, persistence, outbox
`nexus/platform/schema.py`	Tables, `initialize`, `upsert_frame`	Keys, bulk load, schema concurrency
`nexus/platform/data.py`	`ingest_m5`, `ingest_public`	Validation and provenance
`nexus/platform/models.py`	`features`, `recursive_forecast`, `train_demand`, `train_risk`	Leakage, evaluation and inference
`nexus/platform/intelligence.py`	`retrieve`, `optimize_procurement`, `build_graph`, `plan_replenishment`	Retrieval, optimization and orchestration
`nexus/platform/simulator.py`	`initialize_scenario`, `simulate`	Synthetic entity mapping and seeded events
`nexus/platform/streaming.py`	`publish_pending`, `process_event`, `worker`	Messaging and recovery
`nexus/platform/execution.py`	`execute_decision`	Approval, revalidation and idempotence
`nexus/platform/routes.py`	Platform endpoints	UI-to-platform contract
`components/ControlTower.jsx`	Navigation, forms and operational views	Frontend state and user journeys
`components/PlatformWorkspace.jsx`	Forecasts, simulation and monitoring	Advanced platform UI
`tests/test_platform.py`	Scenario fixture and invariant tests	Testing strategy
`tests/test_workflows.py`	Authentication and workflow tests	API correctness

File	Read these parts	Interview concept
`docker-compose.platform.yml`	Services, ports, volumes	Deployment topology

`nexus/domain.py` contains the earlier baseline forecasting and rule-based disruption analysis. `main_research.py`, the research requirements/Compose files and original experimental modules are not the active platform path. Start code explanations from the actual imports and launch command rather than assuming every file in the repository runs.

Trace exercise A

Start at the simulation button in `PlatformWorkspace.jsx`. Follow the request to the route, `simulate`, `Store.enqueue`, `publish_pending`, `process_event`, `plan_replenishment`, and `build_graph`. For every boundary, say whether the call is synchronous Python, HTTP, a database operation, or Kafka transport.

Trace exercise B

Start at decision approval. Find the session/role check, the pending-state check, the database approval update and queued event. Then locate the consumer's `decision.approved` branch and executor. Identify exactly where the purchase order, shipment, decision state and resulting outbox row commit together.

Trace exercise C

Find the exact feature-building code that shifts before rolling. Explain why applying a rolling mean first and shifting incorrectly across combined stores could leak data or mix series. Then find recursive forecasting's `values.append(value)` and explain its role.

26 Interview question bank

Back to contents

Try answering each question aloud before revealing the suggested answer. These are technically accurate answer patterns, not a script to recite word for word. Expect follow-ups that ask you to show the relevant code or calculate an example.

Product and architecture questions

▼ 1 What is NEXUS and what problem does it solve

NEXUS is a supply-chain intelligence simulator that connects historical demand forecasts and delay-risk predictions to constrained replenishment plans, approvals and persistent simulated execution. It addresses the gap between seeing an operational problem and taking a controlled, traceable action. It demonstrates this workflow locally; real commercial benefits have not been measured.

▼ 2 Why is it more than a dashboard

The dashboard is the interface. Behind it, events change state, models support planning, an optimizer produces allocations, policies gate execution, and execution emits new events that trigger replanning. The persistent feedback loop is the central distinction. Show an order and its execution/replan history rather than only charts.

▼ 3 What is real and what is simulated

Historical source data, model training/evaluation, PostgreSQL transactions, Kafka messages and policy enforcement are real computations. The future operating scenario and ERP/carrier effects are synthetic. No real purchase is submitted. Separate source histories are not falsely treated as a single company's records.

▼ 4 Why use both PostgreSQL and Kafka

PostgreSQL stores authoritative current state and transactions. Kafka carries events that consumers process and can replay. Their responsibilities differ. The outbox bridges the database/message boundary, and the inbox prevents duplicate effects. A smaller deployment could use a simpler job queue; Kafka is a deliberate architectural learning tradeoff here.

▼ 5 Is this a microservices system

It has separately deployed services and a separate event-worker process, but much of the business logic shares one Python package and one operational database. I would describe it as a modular application with an event-driven worker and supporting services, rather than claim fully independent domain microservices with isolated data ownership.

▼ 6 What is the most important design choice

Connecting business state and outgoing events transactionally, then making consumed effects idempotent, is a strong choice to discuss. It prevents a demonstration from appearing correct only when every service stays healthy. Another defensible choice is the explicit separation of source data from scenario entities. Explain

one choice deeply rather than declaring all choices equally important.

7 What is the largest current limitation

The operating model uses aggregate units and synthetic commercial constraints, so it is not yet a company-specific SKU purchasing system. It also lacks live ERP/carrier integration, robust user administration and production-scale concurrency. The strongest answer connects a specific limitation to a concrete next implementation step.

8 Can a company upload its own data today

Users can enter operational records and add document text; a legacy daily-demand API/import path also exists. There is no complete guided company-data onboarding workflow that remaps products, validates all relationships, retrains advanced models and switches the whole planning system. The M5/DataCo pipeline remains configured for those source schemas.

Data and machine learning questions

9 Did you train the model on eleven million rows

No. The foundation stores 11,128,850 product-day observations for the selected 365-day window. The demand model uses all 1,913 days aggregated to ten stores, giving 19,130 store-day observations before partitioning and feature warmup. Distinguishing stored data scale from training scale is essential.

10 Why did you keep the datasets separate

M5, DataCo and the San Francisco extract do not establish shared company/product/supplier identities. Joining on superficially similar labels would fabricate relationships. Namespaced IDs preserve identity, and scenario mappings are explicitly synthetic. DataCo provides transferable patterns, not proof that its shipments belong to M5 stores.

11 Why chronological splitting rather than random splitting

Forecasting predicts later observations using earlier information. A random split can train on future periods relative to validation rows and distort the deployment problem. Chronological partitions preserve the direction of time. Random splitting can also leak correlated order lines in risk prediction, so that model aggregates to whole orders first.

12 How do you prevent leakage in rolling features

Features are grouped by store and the target series is shifted before a rolling mean or standard deviation is calculated. Thus the target day's actual demand is not an input. Evaluation recursively appends predictions rather than feeding unknown future actuals into later forecast steps. These mechanisms address different leakage paths.

13 Why use a seasonal naive baseline

Weekly demand patterns make a seven-day seasonal baseline a meaningful low-complexity comparison. It tells us whether the boosted model improves over simply repeating the previous week. The implementation selects using validation WAPE and would use the baseline for future forecasts if it won that comparison.

14 Why gradient boosting rather than an LSTM

The implemented problem becomes tabular regression after lag and calendar feature engineering. Gradient boosting can capture nonlinear interactions without a large sequence-model training setup. The measured result is against a seasonal baseline, not against an LSTM. I would not claim LSTMs are worse without a fair chronological comparison and matched tuning budget.

15 Is 5.92 percent WAPE the same as 94.08 percent accuracy

No. WAPE is aggregate absolute regression error divided by aggregate actual demand. Classification accuracy counts correct labels. Subtracting WAPE from 100 does not create a standard classification accuracy measure. State the metric, grain, holdout dates and baseline directly.

16 What exactly improved by about forty five percent

Relative final-holdout WAPE fell by about 44.9%, from approximately 10.7451% for the baseline to 5.9211% for the candidate. The difference is about 4.824 percentage points. It is not a measured 45% reduction in inventory costs, stockouts or total forecast error under all conditions.

17 What if actual demand is zero

Individual zero-demand rows still contribute absolute error to WAPE's numerator. If total absolute actual demand is zero for the entire evaluation set, WAPE is undefined and the code returns None. MAE and RMSE remain computable. A report should not silently replace undefined WAPE with zero.

18 What does recursive forecasting mean

The model predicts one step, appends that prediction to the available history and uses it for later lags and rolling inputs. This avoids using future ground truth during a multi-day forecast. The tradeoff is error propagation. Direct multi-horizon methods are a reasonable future comparison but are not implemented here.

19 Did the final test influence model selection

Selection between candidate and baseline uses only the earlier validation WAPE. The final test reports both methods without changing selection. However, the empirical future range width is calculated from final-holdout errors, so that interval width does use the test outcomes and does not have separately verified coverage. Keep those two statements distinct.

20 Are the prediction intervals statistically guaranteed

No. They are a fixed empirical width based on the selected model's holdout absolute errors, with a nonnegative lower bound. The code explicitly disclaims a coverage guarantee. A stronger design would reserve separate calibration data, inspect coverage by horizon/store and test the resulting ranges on a later untouched period.

21 Why refit the demand model after testing

After fixing the evaluation and selection, more historical observations can be used to fit the model that generates future predictions. The saved test metric still describes the earlier evaluation procedure, not a new in-sample score of the full-history model. Evaluation and the final serving artifact are related but different objects.

22 Is the delay classifier a supplier risk model

Its target is delivery delay in historical order data. It does not measure bankruptcy, credit risk or broad supplier failure. The planning workflow applies it to synthetic supplier shipment features, with domain-shift limitations. The UI's manually assessed supplier risk scores are also distinct from this trained probability.

23 Why exclude actual shipping duration from risk features

Actual shipping duration is known after the outcome and would reveal or closely encode delay. A decision-time model must use information available when making the decision, such as scheduled days and order characteristics. Extremely high performance from outcome-derived features would be misleading rather than impressive.

24 What does ROC AUC of 0.733 mean

It describes how well the model ranks positive versus negative cases across thresholds. Informally, it is the probability of assigning a higher score to a randomly selected positive than a negative, with the usual treatment of ties. It does not mean 73.3% of orders are correctly classified at the deployed threshold.

25 Why report Brier score as well as accuracy

Planning uses probabilities, so a metric that evaluates probability forecasts is valuable. Brier score averages squared probability errors, whereas accuracy only checks which side of a threshold a prediction falls on. A confidently wrong forecast is penalized strongly by Brier score. Calibration plots and decision-cost evaluation would still be useful additions.

26 What happens when the data distribution changes

Historical model performance may no longer transfer. The current simulation has a rolling demand-error indicator, but it does not implement comprehensive drift detection or automatic retraining. A real deployment should monitor feature/label changes, missingness, calibration and performance by segment, then validate a retrained model before activating it.

Agents retrieval and optimization questions

▼ 27 Are these autonomous LLM agents

No. They are named tool-based steps in a fixed LangGraph workflow. The nodes call trained models, retrieval, inventory logic and optimization; no LLM selects the actions. I would describe the implementation as explicit agent-style orchestration and avoid implying autonomous generative reasoning that the code does not perform.

▼ 28 Why use LangGraph if the workflow is sequential

It provides explicit state contracts and identifiable nodes that can be extended with branching and checkpoints. For this simple graph, ordinary Python functions could also work and would reduce dependencies. The current benefit is workflow organization and inspectability; durable graph checkpointing and parallel agent execution are not configured.

▼ 29 Does retrieval change the approval policy

No. Retrieved documents are untrusted supporting context. Structured JSON policy controls permissions and thresholds. The retrieval result is attached as evidence/explanation, not executed as instructions. A text document claiming unlimited purchasing authority cannot change the actual executor's limits.

▼ 30 What vector database and LLM are used

Neither is required in the active retrieval pipeline. It computes TF-IDF vectors in memory, compares them with cosine similarity and returns cited excerpts. This is an extractive retrieval implementation. Semantic embeddings, a vector index and LLM synthesis are possible enhancements, not existing components.

▼ 31 Why not choose the cheapest supplier directly

The cheapest supplier may have insufficient capacity or a larger delay-risk penalty. The solver considers all suppliers under quantity and budget constraints. Its objective favors cost adjusted by a risk heuristic. A direct cheapest-first rule could be adequate in special cases, but optimization makes the stated constraints explicit and extensible.

▼ 32 Is the optimization objective expected monetary loss

Not as currently written. Cost times one plus delay probability is a risk-weighted objective, but there is no estimated monetary cost per delay. To claim expected loss, I would need a defensible consequence model, such as units short multiplied by an estimated shortage cost under arrival scenarios, and validate those assumptions.

▼ 33 What if no feasible procurement plan exists

The optimizer returns infeasible and the planner blocks execution with a reason. It should not violate budget/capacity to produce an apparently successful order. A future operator workflow could offer explicit alternatives such as increasing the budget, reducing the target or reviewing new suppliers; those changes require authorized choices.

34 Does feasible always mean optimal

No. A feasible solution satisfies constraints. An optimal solution has the best objective under the model and solver proof criteria. The solver accepts both states and records whether optimality was established. Even mathematical optimality only applies to the simplified model, not every unmodeled business concern.

35 How do you avoid repeated replenishment orders

The planner first checks for an unresolved purchase-order decision for the warehouse. It also counts inbound shipments when calculating the shortfall. The executor is idempotent for an already executed decision. These controls address different duplication paths, but do not substitute for a company-wide uniqueness and reconciliation design.

36 What does planning confidence of 0.9 mean

It is assigned heuristically when optimization is feasible. It is not a probability calibrated against historical planning success. The trained supplier delay probability is a different quantity. I would label the heuristic clearly and use validated uncertainty or deterministic eligibility criteria in a production approval design.

Distributed systems and backend questions

37 What problem does the outbox solve

It prevents a database state change from committing without a persistent record of the event that should be published. The state and outbox row share a transaction. Publishing happens later, so broker outages do not require discarding the business change. Duplicate publication remains possible and is handled downstream.

38 What happens after a crash before offset commit

If the database transaction already committed, the event's inbox ID also committed. Redelivery returns DUPLICATE without applying the action again, then the offset can be committed. If the database transaction had not committed, it rolls back and the message can be processed normally later.

39 Can you claim exactly once processing

The precise claim is at-least-once message transport with idempotent database effects for an event ID, plus execution idempotence for a decision ID. Kafka producer idempotence does not create an atomic transaction spanning PostgreSQL and Kafka. Real external APIs would need their own idempotency and reconciliation strategy.

40 What happens if Kafka is down

Pending events remain in the PostgreSQL outbox with attempt/error information, and the publisher retries when the broker returns. The real-stack fault test verified recovery. The current publisher can hold a global store lock while waiting on broker acknowledgement, so API responsiveness during an outage is a known design tradeoff.

41 Is Kafka ordering global

No. Ordering is within a partition. NEXUS uses one partition per topic, but messages across simulation, execution and state topics do not have one universal order. Per-warehouse tick validation prevents applying older simulation ticks. Adding partitions requires carefully selecting entity keys and defining cross-topic consistency expectations.

42 Why keep dead letters

They retain payload and error information for events that cannot safely complete. Invalid events are handled directly, while other failures receive bounded attempts before dead-letter persistence. An authorized retry can queue another event after the cause is corrected. A dead-letter queue is evidence and recovery support, not automatic correction of bad business data.

43 Why use a global advisory lock

It preserves simple serialized state-mutation behavior across API and worker processes, reducing race conditions in a prototype with generic records. The downside is contention, especially when graph execution and publication happen inside transactions. The next step is finer-grained locking and relational invariants, not simply removing the lock.

44 How would you scale the worker

First measure the bottleneck. Then partition events by a stable entity key, redesign global locking, ensure per-entity idempotency and ordering, optimize artifact loading, and separate publishing from long business transactions. Multiple workers sharing a one-partition topic cannot all consume that partition concurrently. Scaling the process count alone is insufficient.

45 What happens if capacity changes after approval

Execution recomputes destination capacity including stock and inbound commitments. If the proposed allocation no longer fits, it raises an error and the transaction rolls back. Approval authorizes an intended action but does not freeze external business conditions. A more complete recovery UI would help replace or re-review stale plans.

46 Is the system event sourced

Not in the strict sense. It stores current operational records directly, alongside audit events and Kafka messages. It does not implement reconstruction of every current record solely by replaying an immutable domain-event history. I call it event-driven with transactional state, not fully event-sourced.

47 Why not use JSON for every table

Historical demand has stable relational keys and high row counts, making explicit schema and foreign keys useful. Flexible operational records use serialized JSON for faster prototype development, but this weakens query/index and database relationship support. A mature product would normalize critical order and inventory structures rather than treating all data equally.

Security deployment and evidence questions

▼ 48 Are you using JWT authentication

No. The application issues an opaque random session token, stores its hash and expiry server-side, and uses an HttpOnly SameSite Strict cookie. Server-side sessions make revocation straightforward. The password is separately stored as a salted scrypt hash. Token hashing and password hashing serve different purposes.

▼ 49 Can a manager approve a high value executive plan

The review endpoint checks the plan's required role and rejects a MANAGER when executive approval is required. ADMIN or EXECUTIVE can approve. A regression test verifies the rejection and unchanged pending status. This does not imply a complete role-administration UI or separation of proposer and reviewer.

▼ 50 Is Docker enough to call the project production ready

No. Containers make setup and dependency isolation easier. Production readiness also requires representative data, capacity testing, secure exposure, secrets management, backup schedules, incident response, real integration contracts and clear operational ownership. The verified scope is a local Docker simulation.

▼ 51 What does your health endpoint prove

It verifies the API can perform a simple database query and reports the selected storage/mode. It does not certify Kafka consumption, model freshness, retrieval quality or business correctness. Worker heartbeat and queue checks provide separate signals, and end-to-end checks provide stronger workflow evidence.

▼ 52 What tests are most valuable here

Tests of invariants at failure boundaries: no double stock deduction, no unapproved execution, atomic rollback on capacity failure, chronological feature behavior, order-aware splitting, and recovery after messaging failures. UI tests show workflows are usable, but correctness of these state transitions is more important than merely rendering a page.

▼ 53 How did you verify recovery rather than just assume it

The project contains a fault-injection script that stops its broker, queues simulation events, observes pending retryable outbox entries, brings Kafka back and waits for consumption. It then restarts the worker and verifies resumed processing. A separate restore test rebuilds a backup in a disposable database and checks the 11,128,850-row demand count.

▼ 54 What does a passing restore count not prove

It does not prove every application invariant, every file artifact, production recovery time or an automated backup policy. It establishes that the tested backup restores and the expected demand count is present. A broader recovery plan would check key relationships, operational records, MLflow state, application startup and documented recovery objectives.

55 How do you monitor model drift

The current implementation calculates rolling WAPE over the latest 100 simulation observations and flags a heuristic threshold. That is limited error monitoring, not comprehensive drift detection. A real deployment needs labeled outcomes, segment-aware performance, input-distribution checks and a controlled retraining process. Normal simulation variability may not trigger the configured alert.

56 What security work remains

Production identity/admin workflows, tenant isolation, secrets management, HTTPS exposure, service authentication, dependency maintenance, rate limiting beyond login and a formal security assessment. The saved report distinguishes a zero-finding production frontend dependency scan at its verification time from unresolved development-tooling advisories. A historical scan is not a permanent security guarantee.

Ownership and improvement questions

57 What would you build next

A company-owned data model and guided import pipeline, followed by product-level stock and retraining on uploaded demand. This enables the rest of the system to operate on coherent company records. I would then implement one sandbox ERP/carrier adapter with idempotency and reconciliation before enabling any live action.

58 What would you simplify

For a small single-company tool, I might initially use a simpler background queue and ordinary functions instead of Kafka and LangGraph. For this project's goals, those components make event reliability and orchestration visible. Simplification should follow operational requirements rather than removing technology just to reduce the stack count.

59 How did AI assistance affect the project

Give an honest personal answer. A defensible pattern is: “I used AI assistance for implementation and debugging, and I am validating my understanding by tracing the code, testing invariants and explaining the tradeoffs. Here is a specific component I can walk through and a limitation I identified.” Do not invent independent contributions or claim understanding that you cannot demonstrate.

60 What measurable business impact did you achieve

No real customer financial impact has been established. The measured technical outcomes are the holdout forecasting improvement and verified workflow/recovery behavior. Reduced shortages or costs are intended benefits requiring a representative operational trial, cost assumptions and a suitable comparison design. This distinction demonstrates sound judgment.

27 Seven day learning plan and mock interview

Back to contents

Day one Understand the business story

Read Chapters 2-6. Draw the system from memory and explain it to someone without mentioning library names for the first minute. Define stock, inbound, lead time and replenishment. Deliverable: a sixty-second explanation and a list of five things the project does not claim.

Day two Understand the data

Read Chapters 7-8 and the schema/import functions. Calculate the 11.1 million-row figure. Explain the model's smaller store-day training grain. Inspect a source manifest without exposing credentials. Deliverable: a schema sketch and an explanation of why the three datasets cannot be joined as one company.

Day three Master the models

Read Chapters 9-13. Recalculate the metric example and relative improvement. Explain chronological validation, recursion and the risk-model feature exclusions. Deliverable: a two-minute model answer with one methodological limitation and a proposed improvement.

Day four Master planning

Read Chapters 14-17 and 20. Solve the supplier allocation example and walk through the seven nodes. Deliverable: a complete explanation of how a 510-unit shortfall becomes a reviewed order and why replanning avoids a second order.

Day five Master reliability

Read Chapters 18-21. Explain the two crash windows that require outbox/inbox protection. Inspect the corresponding tests. Deliverable: an answer to “why not exactly once?” and a drawing showing the order of database commit and Kafka offset commit.

Day six Practice demonstration and debugging

Read Chapters 22-25 and 28-29. Run the non-destructive checks, rehearse the demo, and practice identifying a failing layer from symptoms. Fault injection changes the running simulation and should be deliberately scheduled. Deliverable: a five-minute demo with a fallback if services are unavailable.

Day seven Mock interview and honest ownership

Answer fifteen questions without revealing their answers, perform the live trace, and complete the readiness score below. Review weak topics rather than memorizing more buzzwords. Deliverable: a clear pitch, a demonstrated workflow, one deep technical explanation, and a candid improvement plan.

Forty five minute mock interview

Time	Interviewer prompt	What a strong response contains
0-3 minutes	Introduce the project	Problem, workflow, measured result, scope
3-8 minutes	Draw architecture	Browser/API, state, events, models, approvals
8-16 minutes	Defend your forecast result	Grain, time splits, baseline, metrics, limitations
16-24 minutes	Explain duplicate delivery	Outbox, inbox, transaction/offset ordering
24-30 minutes	Solve a replenishment example	Inbound, constraints, objective versus cost
30-35 minutes	Explain agents and retrieval	Fixed graph, no LLM, authoritative policy
35-41 minutes	Diagnose an outage	Evidence-based sequence, no destructive shortcuts
41-45 minutes	Describe next steps and contribution	Priorities, honest ownership and proposed changes

Readiness scoring

Score yourself from 0 to 2 on each of ten abilities: explain business purpose; draw architecture; distinguish real/synthetic data; explain data grain; calculate metrics; explain chronology/recursion; solve the optimizer example; trace approval/execution; defend outbox/inbox recovery; state limitations and personal contribution. Zero means unable without notes, one means partial explanation, and two means a clear explanation plus a follow-up answer or code pointer.

A score of 16/20 or more with no zero in the workflow, modeling or reliability topics is a useful self-assessment target-not a guarantee of interview success. Below that, practice the weakest chapters and repeat the mock interview.

28 Demo script and troubleshooting

Back to contents

Five minute demonstration

Before the interview: start Docker and the prepared stack, verify service status, sign in using your own owner account and inspect the worker heartbeat. Do not show `.env.platform`, passwords or session tokens. Confirm trained artifacts exist. Rehearse with the same browser size you plan to use.

Minute one: show Data foundation and explain the three distinct source identities and M5 validation. Say which numbers describe storage and which describe model training. Avoid reading every field aloud.

Minute two: show Forecast lab. State the holdout dates, 5.92% versus 10.75% WAPE, and historical forecast cutoff. Explain one lag feature and the seasonal baseline.

Minute three: run one simulation day. Explain that the request queues ten events and the worker applies them. Show stock/planning changes once processing finishes. There may already be unresolved decisions; the planner deliberately reuses them rather than always creating a new one.

Minute four: choose a pending typed simulation purchase-order plan, inspect its context and approve with a meaningful reason. Show its transition to executed and the created purchase order/shipment. Do not choose a legacy sample narrative disruption plan if you want to demonstrate purchase execution.

Minute five: show agent runs, execution/replanning events and monitoring. Briefly explain inbox deduplication and mention that real broker outage/restore checks are recorded. Finish with one practical next step, such as company-data onboarding.

If something goes wrong

Symptom	First checks	Explanation to give
Page does not load	Web container, port 3000, API health	Separate interface availability from data/model problems
Sign-in returns 401	Credentials and session expiry	Use normal account flow, not database password shortcuts
Simulation is queued but stock unchanged	Worker age, pending outbox, broker and worker logs	Accepted asynchronous work is not completed work
Worker repeatedly restarts	Last error, topic configuration, model availability	Fix the cause before replaying events
Forecast is unavailable	Artifact files and training status	Do not substitute fabricated successful results
Plan is pending	Value, risk, confidence, severity and required role	Pending can be correct policy behavior

Symptom	First checks	Explanation to give
No new plan appears	Existing unresolved decision or sufficient inbound	Deduplication and inventory position may explain it
Approved action fails	Current capacity, policy version, supplier state, dead letters	Revalidation prevents stale approvals from forcing invalid actions
Historical dates appear	M5 source cutoff	Historical evaluation is intentionally not relabelled as current sales

For logs, use `docker compose --env-file .env.platform -f docker-compose.platform.yml logs --tail 60 worker` from the repository root. Inspect errors before retrying. Do not restart every service or delete volumes as a first response.

If the machine or containers are unavailable, use the screenshots and saved reports included with the project and explain that you are showing recorded evidence rather than a live demo. A controlled explanation of failure is better than pretending an inactive screen is updating.

29 Practical exercises with solutions

Back to contents

Exercise one Inventory position

A warehouse has stock 120, inbound 250, forecast 80/day and capacity 1,000. What does the current rule request?

Solution: target `min(1000, 80×7) = 560`; available `120+250 = 370`; shortfall `560-370 = 190`. If inbound were 500, available would be 620 and no additional order would be needed. This simple rule does not explicitly time-phase whether inbound arrives before a stockout.

Exercise two Policy boundaries

Evaluate four feasible plans: A has value $10,000, confidence 0.85, risk 0.49 and MEDIUM severity. B differs only in risk 0.50. C has value $100,000. D has value $5,000,001.

Solution: A can auto-approve. B is pending because risk must be strictly below 0.5. C is pending and requires executive-level approval. D is blocked for exceeding the maximum value. A critical severity would also force human review even when A's other thresholds pass.

Exercise three Duplicate execution

An approved order commits, but the worker crashes before acknowledging the Kafka message. What happens after restart?

Solution: the event's inbox ID committed with its state changes. Redelivery is recognized and does not repeat execution. Separately, the executor also returns an already executed decision if directly retried. Explain why these are two layers of protection, not proof of exactly-once transport.

Exercise four Read a feature expression

What is wrong with computing `rolling(7).mean()` on the unshifted target and using it to predict that same day's target?

Solution: the average contains the answer being predicted. Shift first, group by the correct series, and ensure later evaluation steps do not substitute future actuals for predictions. An out-of-time split alone cannot repair a leaked feature.

Exercise five SQL data validation

Explain these read-only SQL examples. They are teaching queries for PostgreSQL and may scan large tables; use them deliberately.

```
-- Product-day rows without a matching series should be absent.
SELECT COUNT(*) AS orphan_count
FROM m5_demand d
```

```
LEFT JOIN m5_series s ON s.series_id = d.series_id
WHERE s.series_id IS NULL;

-- Count available history per store.
SELECT store_id, COUNT(*) AS days, MIN(date), MAX(date)
FROM m5_store_daily
GROUP BY store_id
ORDER BY store_id;

-- Inspect delivery backlog without exposing credentials.
SELECT status, COUNT(*) AS events, MAX(attempts) AS max_attempts
FROM outbox
GROUP BY status;
```

Solution: the first checks a referential invariant, the second checks coverage and expected grain, and the third identifies messaging backlog/retries. A zero orphan count does not establish correct demand values; different validation checks answer different questions.

Exercise six Explain a code-level invariant

Find the transaction containing purchase-order insertion and `execution.completed` enqueue. Then explain what would break if enqueue occurred outside the transaction after it committed.

Solution: a process crash could leave a purchase order with no durable instruction to notify downstream consumers. Replanning might never happen. The outbox row must commit atomically with the state change; actual network publication can happen later.

Exercise seven Design a real ERP adapter

Propose the minimum contract for replacing simulated purchase-order execution. Do not change the live project during an interview without authorization.

Solution: include an external correlation/idempotency key, typed product/vendor mapping, sandbox/live configuration, credential handling, request/response persistence, submission and confirmation states, uncertain-outcome reconciliation, rate limits and retry policy. A network timeout may mean the ERP accepted the order but the response was lost; blindly creating a new order is unsafe. Persist intent first, reconcile by idempotency/correlation key, and only mark confirmed success when evidence exists.

Exercise eight A safe forecast improvement experiment

Design a comparison adding holiday/price features without contaminating the final test.

Solution: verify those features are available at the intended prediction time; create training and validation windows; tune/select using validation or rolling-origin folds; lock the candidate and compare once on a later untouched test. Maintain the same forecast horizon and grain. Do not repeatedly inspect the original test while tuning and continue calling it untouched.

Exercise nine STAR without invented experience

Prepare an answer to “Tell me about a difficult bug.” Use Situation, Task, Action and Result, but use only your actual role.

Solution pattern: “During restart verification, the worker failed when all topics already existed. The task was to make initialization safe to repeat. The implementation was changed to skip an empty create-topics call, and a regression test covered the existing-topic case. A later restart recovery check passed.” If you studied this fix rather than personally authored it, say that. Then explain the code and why repeated startup must be idempotent.

30 Final revision sheet and claim checklist

Back to contents

Numbers to remember

Fact	Value
M5 products and stores	3,049 products; 10 stores
M5 product/store series	30,490
Imported product-day window	365 days; 11,128,850 rows
Store-level history	1,913 days; 19,130 store-days
Forecast horizon	28 days; 280 final test store-days
Validation dates	29 February-27 March 2016
Final test dates	28 March-24 April 2016
Demand WAPE	Candidate 5.92%; baseline 10.75%
Demand MAE and RMSE	250.30 and 349.81 units
Relative WAPE reduction	About 44.9%
DataCo records	180,519 lines; 65,752 aggregated orders
Risk evaluation	AUC about 0.733; Brier about 0.196; accuracy about 69.27%
SF procurement extract	20,000 accounting rows
Graph structure	Seven sequential nodes
Kafka topics	Four; worker consumes three; one partition per topic
Approval thresholds	$10k auto ceiling; 0.85 confidence floor; risk below 0.5; $100k executive threshold; $5m max
Recorded tests	42 backend; 3 browser workflows; separate live-stack/recovery/restore checks

Claims to make precisely

- Say **at-least-once messages with idempotent database effects**, not distributed exactly once.
- Say **store-level forecasting**, not a trained model for every SKU.
- Say **tool-based LangGraph workflow**, not autonomous LLM agents.

- Say **TF-IDF retrieval with cited extracts**, not an embedding/vector-database generative system.
- Say **simulated ERP execution**, not real supplier purchases.
- Say **about 45% lower holdout WAPE**, not 45% business cost savings.
- Say **empirical ranges**, not proven 90% future coverage.
- Say **independent historical datasets**, not one company's integrated production records.
- Say **configured CI and recorded local checks**, not a remote CI run you have not inspected.
- Say **local Docker deployment**, not globally available production hosting.

A strong closing answer

“The project demonstrates that model predictions can be connected to controlled, observable operational actions. The most important engineering work is preserving trustworthy data, evaluating forecasts in time order, enforcing constraints and approvals, and handling retries without duplicate effects. The next step is coherent company-owned product data and a sandbox-tested business-system adapter.”

31 Glossary for quick reference

Back to contents

Term	Meaning in this project
Aggregate	A summary, such as total daily units across all products in a store
Artifact	Saved model, forecast file, evaluation JSON or other training output
Atomic transaction	Related changes commit together or roll back together
Backtest	Evaluate predictions on a historical period as though its outcomes were not yet known
Baseline	Simple reference method used to judge added model value
Calibration	Agreement between predicted probabilities/ranges and observed frequencies
Chronological split	Earlier data trains the model; later data validates/tests it
Consumer offset	Position recording progress within a Kafka partition
Constraint	A rule every feasible optimization solution must satisfy
Data grain	What one row represents, such as product/day or whole order
Data lineage	Trace of origin, transformations and model/run usage
Dead letter	A retained event that could not be processed successfully
Domain shift	A difference between model training conditions and application conditions
Drift	Change in a distribution or performance over time; not every error spike proves it
Feature	An input used by a predictive model
Feasible solution	A solution satisfying the modeled constraints
Foreign key	Database reference that links a row to an existing related row
Heuristic	A practical rule or approximation without a claim of exact probabilistic meaning
Idempotent operation	Repeating the same identified operation does not repeat its intended effect
Inbound	Stock expected from undelivered shipments
Inference	Using a fitted model to produce a prediction
Lag	A value from an earlier time step
Leakage	Use of information unavailable at the intended prediction time

Term	Meaning in this project
Lead time	Time from an order or dispatch to receipt, depending on the defined process
Namespace	Origin/type prefix that separates otherwise similar identifiers
Objective	Quantity an optimizer minimizes or maximizes
Optimal solution	Best solution under the modeled objective and constraints
Outbox	Persisted events awaiting publication, written with business state
Inbox	Record of event IDs already applied by a consumer
Partition	Ordered subdivision of a Kafka topic
Policy	Authoritative rules controlling permitted actions and approvals
Provenance	Evidence about where data or a result came from
Reconciliation	Comparing internal and external state to resolve uncertain or inconsistent outcomes
Recursive forecast	Predict a step and use that prediction to construct later inputs
Replenishment	Ordering/transferring stock to restore an inventory target
ROC-AUC	Ranking discrimination measure for binary outcomes
Rolling feature	Statistic calculated over a moving historical window
Session	Server-recognized authenticated login state
SKU	Specific stock-keeping product identifier
Source of truth	The authoritative persisted representation used for decisions
TF-IDF	Text weighting based on within-document frequency and cross-document rarity
Tick	One discrete simulation time increment
Upsert	Insert a new key or update the row already holding that key
WAPE	Total absolute forecast error divided by total absolute actual demand

32 Evidence and scope of this guide

Back to contents

This guide was checked against active source files including `nexus/api.py`, `nexus/store.py`, the modules under `nexus/platform`, `policies/approval.json`, the active UI components, deployment configuration and test definitions. Numerical results come from `runtime/artifacts/demand_model.json`, `risk_model.json`, `m5_validation.json`, `public_sources.json` and the saved verification reports. The original development history supplies the described observed fixes.

The guide is self-contained for studying concepts and answers. File references let you substantiate them in a code review, but the explanations do not require opening external articles. Dates, thresholds and measured values describe this version; if you modify the code or retrain on different data, update the guide rather than continuing to repeat obsolete figures.

Your final mastery test is practical: explain the architecture without reading, calculate one model metric and one allocation, locate the relevant functions, demonstrate one controlled execution, and describe a failure/recovery path without overclaiming. That is the preparation this handbook is designed to support.

NEXUS interview study edition · Explain what the code does, support what you claim, and practice what you demonstrate.